1.Text Description

Overview

The backend is a single Flask 3.0 process that serves all features of the application including story #1 (Outreach Message Guidance) and Story #7 (Outreach Readiness Check) which are in backend/app.py and share a business-logic module(outreach.py). The architecture has five horizontal tiers: the SPA frontend, an Auth layer, the Flask HTTP routing layer, the shared business-logic module, and a split data tier consisting of a SQLite database for mutable data and static Python modules for read-only reference data. Dependencies flows downward.

Frontend SPA (app.html / api.js)

The frontend is HTML/JS/CSS SPA. All backend communication goes through api.js, which provides two methods for the outreach features:

- generateOutreachMessage(recipientId, tone, goal, customNote) — calls POST /api/outreach/generate
- getOutreachReadiness(userId?) — calls GET /api/outreach/readiness

All authenticated requests include a Bearer token in the Authorization header, which the backend resolves in the auth layer.

Auth Layer (database.py sessions)

Two new endpoints handle user identity:

- POST /api/auth/login — verifies email/password, returns a session token
- POST /api/auth/register — creates a new user account, returns a session token

Session tokens are stored in an in-memory dict inside database.py. Every protected route calls _auth_user(), which reads the Authorization header, resolves the token to a user ID, and fetches the full user dict from SQLite. If no token is present, it falls back to user id=1 for backward compatibility.

Flask HTTP Layer (backend/app.py)

app.py handles routing, input validation, and auth resolution. It never computes business logic directly. For the two outreach routes:

- POST /api/outreach/generate — validates recipientId (required, positive int), sanitizes tone/goal/custom_note via outreach.sanitize_text(), then delegates to outreach.generate_outreach_message()
- GET /api/outreach/readiness — resolves user from userId param or current session, then delegates to outreach.compute_outreach_readiness()

Validation errors return 400; unknown users return 404. All other existing routes (feed, jobs, messaging, notifications) are unchanged.

Business Logic Module (backend/outreach.py)

outreach.py is a pure-function, stateless module exposing two public functions and one shared sanitization utility:

- generate_outreach_message(sender, recipient, context) → dict — builds a personalised draft using tone/goal templates, injects detail fields, and returns draft, char_count, tips, and alternatives

• compute_outreach_readiness(user) → dict — runs 9 weighted evaluator lambdas against a user dict, returns score, level, breakdown, top_tips, and can_message • sanitize_text(value, max_length) — strips HTML tags and control characters from user input before it reaches templates; called by routes in app.py

The two functions share internal helpers (_first_name, _TONE_OPENERS, _TONE_CLOSERS, _GOAL_LINES, _OUTREACH_CHECKS) with no duplication. The AI abstraction point _call_ai() is also in this module — in P4 it is an identity function; in P5 it is replaced with a boto3 Bedrock call without touching any other file.

Data Layer (database.py + data/*.py)

The data tier is split into two parts:

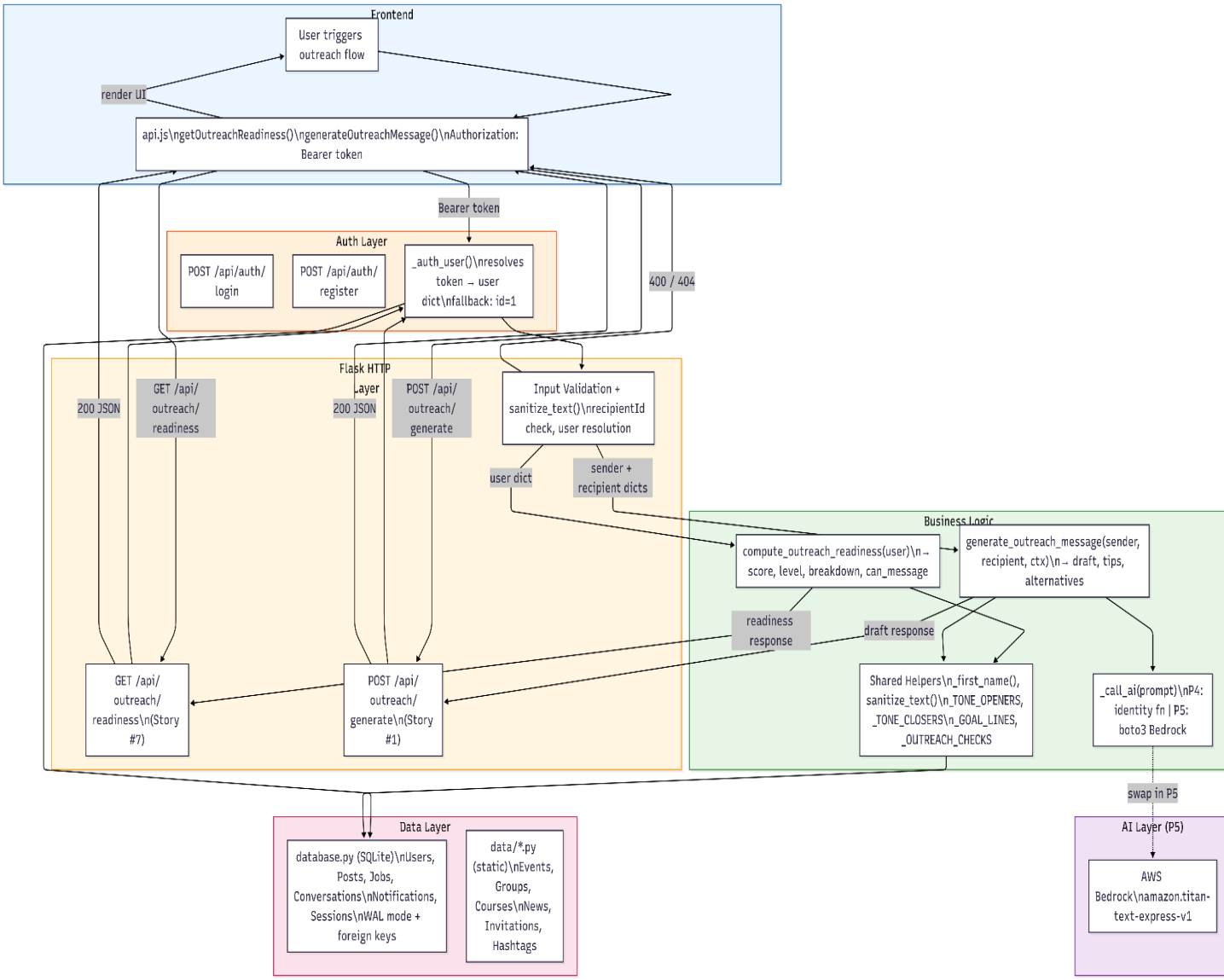
- database.py (SQLite) — stores all mutable data: users, posts, jobs, companies, conversations, messages, notifications. Uses WAL mode and foreign key enforcement. Seeded from data/*.py on first init. Also owns the session store.
- data/*.py (static) — read-only reference data (events, groups, courses, news, invitations, hashtags) served directly by app.py routes without going through database.py.

outreach.py reads user data exclusively through dicts passed in by app.py — it never imports database.py or data/*.py directly, preserving its stateless, testable character.

Dependency Relationship

Story #7 (Readiness Check) remains a prerequisite for Story #1 (Message Generator). The frontend calls GET /api/outreach/readiness first; if can_message is false (score < 60), the user sees improvement tips before the message form is enabled. Both endpoints share the same _auth_user() resolution logic and the same sanitize_text() helper in outreach.py.

2. Architecture Diagram (Mermaid)



If not clearly visible use this link: https://mermaid.ai/app/projects/

Diagram summary: Six colour-coded subgraphs. The blue Frontend passes Bearer tokens to the new orange Auth layer (_auth_user()), which resolves them against SQLite. The amber Flask tier validates input and delegates to the green Business Logic tier. The logic tier reads from the pink Data layer (SQLite for mutable data, static Python modules for reference data). A dashed arrow from _call_ai() to the purple AWS Bedrock node marks the P4 → P5 upgrade point.

Code Used:

```
subgraph FE["Frontend (SPA — app.html / api.js)"]
    UI["User triggers outreach flow"]
    API["api.js\ngetOutreachReadiness()\ngenerateOutreachMessage()\nAuthorization: Bearer token"]
end

subgraph AUTH["Auth Layer (database.py sessions)"]
    LOGIN["POST /api/auth/login"]
    REG["POST /api/auth/register"]
    SESSION["_auth_user()\nresolves token → user dict\nfallback: id=1"]
end

subgraph BE["Flask HTTP Layer (backend/app.py)"]
    R7["GET /api/outreach/readiness\n(Story #7)"]
    R1["POST /api/outreach/generate\n(Story #1)"]
    VAL["Input Validation + sanitize_text()\nrecipientId check, user resolution"]
end

subgraph BL["Business Logic (backend/outreach.py)"]
    CR["compute_outreach_readiness(user)\n→ score, level, breakdown, can_message"]
    GM["generate_outreach_message(sender, recipient, ctx)\n→ draft, tips, alternatives"]
    SH["Shared Helpers\n_first_name(), sanitize_text()\n_TONE_OPENERS, _TONE_CLOSERS\n_GOAL_LINES,
_OUTREACH_CHECKS"]
    AI["_call_ai(prompt)\nP4: identity fn | P5: boto3 Bedrock"]
end

subgraph DL["Data Layer"]
    SQLITE["database.py (SQLite)\nUsers, Posts, Jobs, Conversations\nNotifications, Sessions\nWAL mode + foreign keys"]
    STATIC["data/*.py (static)\nEvents, Groups, Courses\nNews, Invitations, Hashtags"]
end

subgraph AICLOUD["AI Layer (P5)"]
    BEDROCK["AWS Bedrock\namazon.titan-text-express-v1"]
end

UI --> API
API -->|"Bearer token"| SESSION
SESSION --> SQLITE
API -->|"GET /api/outreach/readiness"| R7
API -->|"POST /api/outreach/generate"| R1
R7 --> SESSION
R1 --> SESSION
SESSION --> VAL
VAL -->|"user dict"| CR
VAL -->|"sender + recipient dicts"| GM
VAL -->|"400 / 404"| API
CR --> SH
GM --> SH
GM --> AI
AI -.->|"swap in P5"| BEDROCK
CR -->|"readiness response"| R7
GM -->|"draft response"| R1
R7 -->|"200 JSON"| API
R1 -->|"200 JSON"| API
API -->|"render UI"| UI
SH --> SQLITE

style FE fill:#EBF5FF,stroke:#2E75B6,stroke-width:2px
style AUTH fill:#FFF3E0,stroke:#E65100,stroke-width:2px
style BE fill:#FFF8E1,stroke:#F59E0B,stroke-width:2px
style BL fill:#E8F5E9,stroke:#388E3C,stroke-width:2px
style DL fill:#FCE4EC,stroke:#C2185B,stroke-width:2px
style AICLOUD fill:#F3E5F5,stroke:#7B1FA2,stroke-width:2px
```

3. Design Justification

Single Flask Process:

Both stories share user-resolution logic, the same data source, and the same future AI dependency. Splitting them into microservices would have introduced duplicated user lookup, inter-service network calls, and two deployment units with no concurrency benefit. A single Flask process with threaded=True handles concurrent requests safely and keeps operational complexity near zero.

SQLite Replaced In-Memory Dicts:

The original design used in-memory dicts because P4 was a mock phase. During implementation, the team added real user registration and login, which required data to persist across restarts. SQLite with WAL mode was the right choice: zero infrastructure, file-based, supports concurrent reads, and requires no migration from the existing data shapes. The switch was contained entirely inside database.py — outreach.py and app.py saw no change to their interfaces.

Auth Lives in database.py, Not a Separate Module:

Session tokens are just a dict keyed by token string mapping to user IDs — logically they belong with the data layer that owns user records. Putting them in a separate auth.py would have added an import dependency between two modules that both need the same SQLite connection. Colocating them in database.py keeps the connection management in one place.

sanitize_text() Lives in outreach.py, Not app.py:

Sanitization is called by app.py before passing data to outreach.py, but the function itself lives in outreach.py because it is domain-specific: it exists to prevent injection into outreach templates and future LLM prompts. Putting it in outreach.py keeps the security boundary co-located with the code it protects, and makes it available to any future caller that invokes the module directly.

outreach.py Remains Pure Despite the SQLite Upgrade:

outreach.py still receives fully resolved dicts from app.py — it never imports database.py. This was on purpose to preserve testability: the two public functions can be unit-tested by passing a dict directly, with no database setup required. The P5 Bedrock swap still requires touching only the four-line body of _call_ai(), and nothing else.